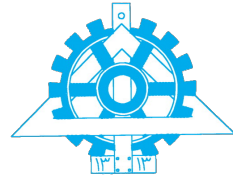




University of Tehran
College of Engineering
School of Electrical and
Computer Engineering



Machine Learning

Assignment 1

Full Name

Sepanta Ghonoodi

Student ID

810102483

November 24, 2025

Abstract

In this homework, we explored a wide range of optimization techniques that are fundamental to machine learning. We started by studying the theory behind line search methods, including how to define descent directions and select appropriate step sizes using strategies like Armijo and Wolfe conditions.

We also solved more complex problems involving bilevel and multi-objective optimization to understand trade-offs in decision-making. The assignment also had a strong practical component where we implemented several algorithms from scratch. We coded and compared Gradient Descent against Newton's Method on the Rosenbrock function, analyzing how techniques like Backtracking Line Search and AdaGrad influence convergence speed and stability. This work gave us a solid understanding of both the theoretical proofs and the implementation details required for efficient optimization.

Contents

Abstract	i
1 Question 1: Line Search Optimization Problem	1
1.1 General Form	1
1.2 Descent Direction	1
1.3 Step Length Methods	2
1.4 Quadratic Function Example	2
2 Question 3: Bilevel Optimization	3
2.1 bilevel formulation	3
2.2 solving y^*	4
2.3 substitution of y^*	4
2.4 finding x^*	5
2.5 Regularity	5
3 Question 7: Batch vs. Stochastic Gradient Descent	6
3.1 Computational Load	6
3.2 Noise	6
3.3 Stability and Convergence	6
4 Question 8: Steepest Descent Direction via First-Order Taylor	7
4.1 First-Order Taylor Series Approximation	7
4.2 Optimization Problem	7
4.3 Finding the Direction	7
5 Question 9: Pseudoinverse Solution is the Least-Squares Minimizer (Optional)	8
5.1 Derivation of the Loss Function	8
5.2 Gradient Calculation & Normal Equations	8
5.3 Hessian & Convexity Proof	9
5.4 Conclusion	9
6 Question 11: Quadratic Convergence of Newton's Method	9
6.1 Quadratic Convergence	10
6.2 Newton's Method Approximation	10
7 Question 13: Equality-Constrained Optimization via Lagrange Multipliers & Newton–Raphson (Optional)	11
7.1 Lagrangian Definition	11
7.2 Optimality Conditions	11
7.3 System Definition	12
7.4 Jacobian Matrix	12
7.5 Newton–Raphson Update	13
7.6 Implementation and Convergence Results	13
8 Question 14: Step-Size Strategies in Gradient Descent	14
8.1 Step Size Selection Strategies	14
References	16

List of Figures

- | | | |
|---|--|----|
| 1 | Optimization trajectories (left) and cost convergence (right) for Armijo vs. Ada-Grad. | 15 |
|---|--|----|

Question 1: Line Search Optimization Problem

- Write the general form of the optimization problem that line-search algorithms solve, and briefly explain the roles of the **search direction** and the **step length**.
- Define a **descent direction** in the context of line search. State the necessary condition for a vector p to be a descent direction at x .
- List at least three common methods to determine the step length α in line-search algorithms. For each method give a short explanation (e.g., exact line search, Armijo/backtracking, Wolfe conditions, fixed step, Barzilai-Borwein).
- Consider the quadratic function

$$f(x_1, x_2) = 2x_1^2 + x_1x_2 + 4x_2^2.$$

At iteration t , the current point and search direction are

$$x^{(t)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad p^{(t)} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}.$$

- Show that $p^{(t)}$ is a descent direction at $x^{(t)}$, i.e. show that $\nabla f(x^{(t)})^\top p^{(t)} < 0$.
- Form the one-dimensional function $\phi(\alpha) = f(x^{(t)} + \alpha p^{(t)})$ and simplify it to exhibit its quadratic form.
- Compute $\alpha^* > 0$ that minimizes $\phi(\alpha)$ (exact line search). State whether $\alpha^* > 0$.

1.1 General Form

This algorithm tries to minimize $\phi(\alpha) = f(x^{(t)} + \alpha p^{(t)})$ for $\alpha > 0$ in which $x^{(t)}$ is the current point, α is the step size and $p^{(t)}$ is the search direction. This algorithm wants to minimize f on a line in direction of $p^{(t)}$ from the point $x^{(t)}$ by finding the correct α .

Search Direction ($p^{(t)}$): This determines which way to move from point $x^{(t)}$. $p^{(t)}$ is chosen to be in descent direction to lower the value of the function.

Step length: A positive value used to determine how much to move in $p^{(t)}$ direction.

1.2 Descent Direction

A descent direction at a point x is a vector p such that moving a small distance from x in the direction of p causes the function value to decrease.

In order for p to be a descent direction it must have (necessary condition) $\nabla f(x)^\top p < 0$.

1.3 Step Length Methods

- **Exact line search:** This method tries to find the exact value of α^* . It is only practical for simple functions. The value of α^* is calculated by setting $\phi'(\alpha) = 0$ and finding α .
- **Armijo backtracking:** This method is not exact, it uses a condition to ensure your step is not too long.
 - Condition: $f(x + \alpha p) < f(x) + c_1 \alpha \nabla f(x)^\top p$
 - We choose a small constant value for c_1 to prevent rapid changes in $f(x + \alpha p)$.
- **Wolfe conditions:** This method uses another condition additional to the Armijo Condition. The Secondary Condition is:

$$\nabla f(x + \alpha p)^\top p > c_2 \nabla f(x)^\top p$$

This condition ensures our steps are not too short.

- **Fixed step:** This is also an inexact method in which we choose a small fixed value for α .

1.4 Quadratic Function Example

$$1) \nabla f(x) = \begin{pmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \end{pmatrix} = \begin{pmatrix} 4x_1 + x_2 \\ x_1 + 8x_2 \end{pmatrix}$$

$$\Rightarrow \nabla f(x)^\top p^{(t)} = [4x_1 + x_2 \quad x_1 + 8x_2] \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

$$= (4x_1 + x_2) - 2(x_1 + 8x_2) = 4x_1 + x_2 - 2x_1 - 16x_2 = 2x_1 - 15x_2$$

$$\text{At } x^{(t)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \Rightarrow 2(0) - 15(1) = -15 < 0.$$

Therefore $p^{(t)}$ is in descent direction.

$$2) \phi(\alpha) = f(x^{(t)} + \alpha p^{(t)}) = f\left(\begin{pmatrix} 0 \\ 1 \end{pmatrix} + \alpha \begin{pmatrix} 1 \\ -2 \end{pmatrix}\right) = f\left(\begin{pmatrix} \alpha \\ 1 - 2\alpha \end{pmatrix}\right)$$

$$\begin{aligned} \phi(\alpha) &= 2(\alpha)^2 + (\alpha)(1 - 2\alpha) + 4(1 - 2\alpha)^2 \\ &= 2\alpha^2 + \alpha - 2\alpha^2 + 4(1 - 4\alpha + 4\alpha^2) \\ &= 2\alpha^2 + \alpha - 2\alpha^2 + 4 - 16\alpha + 16\alpha^2 \\ &= 16\alpha^2 - 15\alpha + 4 \end{aligned}$$

3) To find the minimum, set $\phi'(\alpha) = 0$:

$$\phi'(\alpha) = 32\alpha - 15$$

$$32\alpha - 15 = 0 \Rightarrow \alpha^* = \frac{15}{32}$$

Yes, α^* is positive.

Question 3: Bilevel Optimization

Introduction (short). Bilevel optimization models a leader choosing variables anticipating the follower's optimal response. Below is a simple scalar example solvable analytically: The leader chooses $x \in \mathbb{R}$ to

$$\min_x F(x, y) = (x - 2)^2 + (y - 3)^2 \quad \text{s.t. } x \geq 0,$$

while the follower, given x , chooses $y \in \mathbb{R}$ to

$$\min_y f(x, y) = (y - x^2)^2 + y \quad \text{s.t. } y \geq 0.$$

- Write the bilevel formulation (upper and lower).
- For fixed x , solve the follower and obtain $y^*(x)$.
- Substitute $y^*(x)$ into the leader objective and reduce to one variable.
- Find (x^*, y^*) .
- Discuss regularity (convexity of follower, uniqueness, validity of substitution).

2.1 bilevel formulation

$$\begin{aligned} \min_x \quad & (x - 2)^2 + (y - 3)^2 \\ \text{s.t.} \quad & x \geq 0 \\ & y \in \arg \min_z \{(z - x^2)^2 + z \mid z \geq 0\} \end{aligned}$$

2.2 solving y^*

$$f(y) = (y - x^2)^2 + y \quad \text{subject to } y \geq 0$$

$$\frac{df}{dy} = 2(y - x^2) + 1$$

$$2(y - x^2) + 1 = 0$$

$$2y - 2x^2 + 1 = 0$$

$$y_{unc} = x^2 - \frac{1}{2}$$

now we enforce $y \geq 0$

$$\text{if } x^2 - \frac{1}{2} \geq 0 \Rightarrow \sqrt{0.5} < x$$

$$\text{if } x^2 - \frac{1}{2} \leq 0 \Rightarrow y^* = 0 \quad 0 < x \leq \sqrt{0.5}$$

therefore we have

$$y^*(x) = \begin{cases} 0 & \text{if } 0 \leq x < \sqrt{0.5} \\ x^2 - 0.5 & \text{if } x \geq \sqrt{0.5} \end{cases}$$

2.3 substitution of y^*

We substitute $y^*(x)$ into the leader's objective function $F(x, y) = (x - 2)^2 + (y - 3)^2$. This creates a piecewise function for the leader to minimize:

Region 1 ($0 \leq x < \sqrt{0.5}$):

$$y^* = 0$$

$$F_1(x) = (x - 2)^2 + (0 - 3)^2 = (x - 2)^2 + 9$$

Region 2 ($x \geq \sqrt{0.5}$):

$$y^* = x^2 - 0.5$$

$$F_2(x) = (x - 2)^2 + (x^2 - 0.5 - 3)^2 = (x - 2)^2 + (x^2 - 3.5)^2$$

The reduced one-variable problem is to minimize this piecewise $F(x)$ subject to $x \geq 0$.

2.4 finding x^*

Likewise, we consider the minima in the two regions: In Region 1 ($0 \leq x < 0.707$): $F_1(x) = (x - 2)^2 + 9$ is a down-opening parabola within this segment. Since the vertex occurs at $x = 2$, which is outside this interval, the minimum in our interval must occur at the right endpoint:

$$x \rightarrow \sqrt{0.5} \approx 0.707$$

$$F_1(\sqrt{0.5}) \approx (0.707 - 2)^2 + 9 \approx 1.67 + 9 \approx 10.67$$

for Region 2 we have $F_2(x) = (x - 2)^2 + (x^2 - 3.5)^2$. Take the derivative with respect to x :

$$F'_2(x) = 2(x - 2) + 2(x^2 - 3.5)(2x)$$

$$F'_2(x) = 2x - 4 + 4x^3 - 14x$$

$$F'_2(x) = 4x^3 - 12x - 4$$

by setting the derivative to 0 we get:

$$4(x^3 - 3x - 1) = 0$$

$$x^3 - 3x - 1 = 0$$

$$x = -1.53 \text{ (out of bounds)}$$

$$x = -0.34 \text{ (out of bounds)}$$

$$x \approx 1.879 \text{ (acceptable)}$$

Therefore we can calculate the minimum in the second region as:

$$y^* = (1.879)^2 - 0.5 \approx 3.532 - 0.5 = 3.032$$

We can conclude that the minima in region 2 less than the minima in region 1 and the global minimum will be 3.032 in region 2.

2.5 Regularity

Convexity of Follower: The objective function for the follower is $f(x, y) = (y - x^2)^2 + y$. The second derivative with respect to y $\frac{\partial^2 f}{\partial y^2} = 2$, indicating the function is convex in y . Since $2 > 0$ everywhere, the follower's problem is strictly convex.

Uniqueness: Because the follower's problem is strictly convex over the convex feasible set $y \geq 0$, for every value of x , the optimal follower's response $y^*(x)$ is unique.

Validity of Substitution: Because the lower-level solution $y^*(x)$ is unique (it is a singleton set for every x), the reduction to a single-level optimization problem performed in step (c) is mathematically valid. If the follower had multiple optimal solutions, the leader would have to choose between an "optimistic" or "pessimistic" formulation, and simple substitution would not suffice.

Question 7: Batch vs. Stochastic Gradient Descent

Compare Batch Gradient Descent (BGD) and Stochastic Gradient Descent (SGD). Discuss:

- (a) Computational load per iteration.
- (b) Noise (variance) in the gradient estimation.
- (c) Stability and nature of convergence to the minimum.

3.1 Computational Load

Batch Gradient Descent (BGD): In BGD, we iterate through all data points in each step. This creates a high computational load because we might have millions of samples, and we must calculate the gradient over the entire dataset for every update.

Stochastic Gradient Descent (SGD): In SGD, the computational load per iteration is significantly lower since we estimate the gradient using only a single sample (or a small batch) of the training data, rather than the whole dataset.

3.2 Noise

BGD: BGD has significantly less noise than SGD (technically zero noise in the gradient calculation relative to the cost function) because it takes the entire training set into account. The effect of outliers diminishes because they are averaged out over the large number of data points.

SGD: SGD uses only a single sample (or subset), introducing variance (noise) into the gradient estimation. If the chosen sample is an outlier or unrepresentative, the gradient step may not point in the exact direction of the true descent.

3.3 Stability and Convergence

BGD: BGD ensures convergence to a local minimum (or global minimum for convex problems), and the convergence path is smooth and stable. However, this deterministic path means it may get stuck in a poor local minimum without the ability to escape.

SGD: SGD is more unstable, and the cost function may fluctuate ("go back and forth"). While it might not converge to a specific point with a fixed learning rate (it will oscillate around the

minimum), this noise can actually be beneficial. The stochastic fluctuations help the algorithm jump out of shallow local minima and potentially find a better global minimum.

Question 8: Steepest Descent Direction via First-Order Taylor

Gradient Descent uses the direction $p_k = -\nabla f(x_k)$. Using a first-order Taylor approximation, prove this is the *steepest descent* direction in a local neighborhood of x_k . (Hint: For small $\alpha > 0$, minimize $f(x_k + \alpha p)$ over unit vectors $\|p\| = 1$.)

4.1 First-Order Taylor Series Approximation

We start with the first-order Taylor series approximation of $f(x_k + \alpha p)$ centered at x_k :

$$\begin{aligned} f(x_k + \alpha p) &\approx f(x_k) + \nabla f(x_k)^\top ((x_k + \alpha p) - x_k) \\ &\approx f(x_k) + \alpha \nabla f(x_k)^\top p \end{aligned}$$

Since $f(x_k)$ is a constant (with respect to p) and α is a positive scalar, to minimize the function value, we must minimize the term involving p .

4.2 Optimization Problem

We formulate the minimization problem over unit vectors:

$$\min_p \nabla f(x_k)^\top p \quad \text{s.t.} \quad \|p\| = 1$$

We know that the dot product $\nabla f(x_k)^\top p$ can be written using the geometric definition:

$$\nabla f(x_k)^\top p = \|\nabla f(x_k)\| \|p\| \cos(\theta)$$

4.3 Finding the Direction

Since we constrained $\|p\| = 1$, the expression simplifies to:

$$\|\nabla f(x_k)\| \cos(\theta)$$

To minimize this quantity, we must minimize $\cos(\theta)$. The minimum value of $\cos(\theta)$ is -1 , which occurs when $\theta = 180^\circ$ (or π radians).

This implies that p must be a unit vector in the **opposite direction** of the gradient $\nabla f(x_k)$:

$$p = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}$$

Thus, the direction of steepest descent is the negative gradient, $-\nabla f(x_k)$.

Question 9: Pseudoinverse Solution is the Least-Squares Minimizer (Optional)

For an overdetermined system $Ax = b$ with $A \in \mathbb{R}^{m \times n}$ ($m > n$) full column rank, the candidate

$$x^* = (A^\top A)^{-1} A^\top b$$

satisfies the normal equations $A^\top A x^* = A^\top b$. Prove x^* is the *unique global minimizer* of $L(x) = \|Ax - b\|_2^2$.

5.1 Derivation of the Loss Function

We start by expanding the squared Euclidean norm of the residual:

$$L(x) = \|Ax - b\|^2 = (Ax - b)^\top (Ax - b)$$

Expanding the terms:

$$\begin{aligned} L(x) &= (x^\top A^\top - b^\top)(Ax - b) \\ &= x^\top A^\top Ax - b^\top Ax - x^\top A^\top b + b^\top b \end{aligned}$$

Note that the term $x^\top A^\top b$ is a scalar, so it equals its own transpose:

$$(x^\top A^\top b) = (x^\top A^\top b)^\top = b^\top Ax$$

Substituting this back into the equation:

$$L(x) = x^\top A^\top Ax - 2b^\top Ax + b^\top b$$

5.2 Gradient Calculation & Normal Equations

To find the critical points, we take the gradient with respect to x :

$$\nabla_x L(x) = 2A^\top Ax - 2A^\top b$$

Setting the gradient to zero to find the stationary point:

$$\begin{aligned}\nabla L(x) = 0 &\implies 2A^\top Ax = 2A^\top b \\ &\implies A^\top Ax = A^\top b \\ &\implies x = (A^\top A)^{-1}A^\top b\end{aligned}$$

5.3 Hessian & Convexity Proof

In order to ensure this critical point is a minimum, the Hessian matrix must be positive definite. The Hessian is the second derivative of the loss function:

$$\nabla^2 L(x) = 2A^\top A$$

To prove positive definiteness, we test with an arbitrary non-zero vector v :

$$\begin{aligned}v^\top (2A^\top A)v &= 2(v^\top A^\top)(Av) \\ &= 2(Av)^\top (Av) \\ &= 2\|Av\|^2\end{aligned}$$

Since A has **full column rank**, its columns are linearly independent. Therefore, $Av \neq 0$ for any $v \neq 0$, which implies $\|Av\|^2 > 0$.

Thus, $A^\top A$ is positive definite. Since the Hessian is positive definite everywhere, the function is **strictly convex**.

5.4 Conclusion

Because the function is strictly convex, it has only one global minimum. Therefore, the point $x = (A^\top A)^{-1}A^\top b$ is the **unique global minimizer**.

Question 11: Quadratic Convergence of Newton's Method

- Define *quadratic convergence* using the error $e_k = x_k - x^*$, and contrast with linear convergence.
- Show that if x_k is sufficiently close to a local minimizer x^* , then the Newton direction p_k approximates the exact displacement $(x^* - x_k)$. (Use $\nabla f(x^*) = 0$ and a Taylor expansion of the gradient.)

6.1 Quadratic Convergence

Quadratic Convergence: We define quadratic convergence as:

$$\exists k, \exists c > 0 \quad \text{s.t.} \quad \forall k' > k, \quad \|e_{k'+1}\| \leq c\|e_{k'}\|^2 \quad (I)$$

Based on the question we know $e_k = x_k - x^*$:

$$\Rightarrow \|x_{k+1} - x^*\| \leq c\|x_k - x^*\|^2$$

For Linear Convergence:

$$\exists \mu > 0, \exists k \quad \text{s.t.} \quad \forall k' > k, \quad \|e_{k'+1}\| \leq \mu\|e_{k'}\|$$

$$\Rightarrow \|x_{k+1} - x^*\| \leq \mu\|x_k - x^*\|$$

Contrast: In linear convergence, error is reduced by a constant factor at each step while in quadratic convergence it is reduced much faster because of the square factor in the term.

6.2 Newton's Method Approximation

We use first order Taylor expansion of $\nabla f(x^*)$ since x_k is sufficiently close to x^* :

$$\Rightarrow \nabla f(x^*) \approx \nabla f(x_k) + \nabla^2 f(x_k)(x^* - x_k)$$

We know that x^* is local minimum $\Rightarrow \nabla f(x^*) = 0$:

$$\Rightarrow 0 \approx \nabla f(x_k) + \nabla^2 f(x_k)(x^* - x_k)$$

$$-\nabla f(x_k) \approx \nabla^2 f(x_k)(x^* - x_k)$$

Assuming that the Hessian is non-singular:

$$-\nabla^2 f(x_k)^{-1} \nabla f(x_k) \approx x^* - x_k \quad (I)$$

In quadratic approximation we know:

$$f(x_k + p) \approx f(x_k) + \nabla f(x_k)^\top p + \frac{1}{2} p^\top \nabla^2 f(x_k) p$$

To minimize using p , we take the derivative w.r.t. p and set it to 0:

$$\nabla f(x_k) + \nabla^2 f(x_k) p = 0$$

$$\Rightarrow p = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

Substituting in (I):

$$\Rightarrow p \approx x^* - x_k$$

This shows that if the function is locally quadratic, p_k will attempt to jump to the minimum x^* .

Question 13: Equality-Constrained Optimization via Lagrange Multipliers & Newton–Raphson (Optional)

Solve

$$\min_{x,y} f(x,y) = e^{x^2+y^2} \quad \text{s.t.} \quad g(x,y) = x + y - 1 = 0.$$

- Write the Lagrangian $L(x, y, \lambda)$.
- Find the optimal (x^*, y^*, λ^*) by solving $\nabla_{x,y,\lambda} L = 0$.
- (Coding) Write the 3×3 nonlinear system $F(w) = 0$ with $w = [x, y, \lambda]^\top$.
- Apply Newton–Raphson: $w_{k+1} = w_k - J_F(w_k)^{-1} F(w_k)$. Compute the Jacobian $J_F(w)$.
- Implement Newton’s method and show convergence from $w_0 = [0, 0, 0]^\top$ to the analytic solution from (b).

7.1 Lagrangian Definition

$$L(x, y, \lambda) = f(x, y) + \lambda \cdot g(x, y)$$

Substituting the given functions:

$$L(x, y, \lambda) = e^{x^2+y^2} + \lambda(x + y - 1)$$

7.2 Optimality Conditions

We compute the partial derivatives and set them to zero to find the stationary points:

$$\frac{\partial L}{\partial x} = 2xe^{x^2+y^2} + \lambda = 0 \implies \lambda = -2xe^{x^2+y^2} \quad (I)$$

$$\frac{\partial L}{\partial y} = 2ye^{x^2+y^2} + \lambda = 0 \implies \lambda = -2ye^{x^2+y^2} \quad (II)$$

$$\frac{\partial L}{\partial \lambda} = x + y - 1 = 0 \quad (III)$$

Equating (I) and (II):

$$-2xe^{x^2+y^2} = -2ye^{x^2+y^2}$$

Dividing by the non-zero term $-2e^{x^2+y^2}$, we get:

$$\implies x = y$$

Substituting this result into constraint (III):

$$x + y - 1 = 0 \Rightarrow 2x - 1 = 0 \Rightarrow x = y = \frac{1}{2}$$

Finally, we substitute x and y back into (I) or (II) to solve for λ :

$$\lambda = -2 \left(\frac{1}{2} \right) e^{\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2} = -1 \cdot e^{\frac{1}{4} + \frac{1}{4}} = -e^{\frac{1}{2}}$$

7.3 System Definition

We define the nonlinear system $F(w) = 0$, where $w = [x, y, \lambda]^\top$, using the gradients derived in part (b):

$$F(w) = \begin{bmatrix} \frac{\partial L}{\partial x} \\ \frac{\partial L}{\partial y} \\ \frac{\partial L}{\partial \lambda} \end{bmatrix} = \begin{bmatrix} 2xe^{x^2+y^2} + \lambda \\ 2ye^{x^2+y^2} + \lambda \\ x + y - 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

7.4 Jacobian Matrix

The Jacobian matrix $J_F(w)$ consists of the partial derivatives of $F(w)$:

$$J_F(w) = \begin{bmatrix} \frac{\partial F_1}{\partial x} & \frac{\partial F_1}{\partial y} & \frac{\partial F_1}{\partial \lambda} \\ \frac{\partial F_2}{\partial x} & \frac{\partial F_2}{\partial y} & \frac{\partial F_2}{\partial \lambda} \\ \frac{\partial F_3}{\partial x} & \frac{\partial F_3}{\partial y} & \frac{\partial F_3}{\partial \lambda} \end{bmatrix}$$

Calculating the partial derivatives using the product rule and chain rule:

$$\begin{aligned} \frac{\partial}{\partial x} (2xe^{x^2+y^2}) &= 2e^{x^2+y^2} + 2x(2x)e^{x^2+y^2} = 2(1 + 2x^2)e^{x^2+y^2} \\ \frac{\partial}{\partial y} (2xe^{x^2+y^2}) &= 2x(2y)e^{x^2+y^2} = 4xye^{x^2+y^2} \end{aligned}$$

Assembling the full matrix (noting symmetry due to continuous second derivatives):

$$J_F(w) = \begin{bmatrix} 2(1 + 2x^2)e^{x^2+y^2} & 4xye^{x^2+y^2} & 1 \\ 4xye^{x^2+y^2} & 2(1 + 2y^2)e^{x^2+y^2} & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

7.5 Newton–Raphson Update

Applying the Newton–Raphson iteration formula $w_{k+1} = w_k - J_F(w_k)^{-1} F(w_k)$:

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \lambda_{k+1} \end{bmatrix} = \begin{bmatrix} x_k \\ y_k \\ \lambda_k \end{bmatrix} - (J_F(w_k))^{-1} \begin{bmatrix} 2x_k e^{x_k^2 + y_k^2} + \lambda_k \\ 2y_k e^{x_k^2 + y_k^2} + \lambda_k \\ x_k + y_k - 1 \end{bmatrix}$$

7.6 Implementation and Convergence Results

We implemented the Newton–Raphson method starting from $w_0 = [0, 0, 0]^\top$. The algorithm converged to the analytic solution $w^* = [0.5, 0.5, -1.6487]^\top$ in just two iterations. The results are summarized below:

Iteration (k)	x_k	y_k	λ_k	Error $\ w_k - w^*\ _2$
0	0.0000	0.0000	0.0000	-
1	0.5000	0.5000	-1.0000	6.49×10^{-1}
2	0.5000	0.5000	-1.6487	$0.00 \times 10^{+0}$

Table 1: Convergence of Newton’s Method for the equality-constrained problem.

Analysis of Convergence: The method exhibits extremely fast convergence (effectively in two steps) for the following reasons:

- Linear Constraint:** The constraint $g(x, y) = x + y - 1 = 0$ is linear. Newton’s method solves linear equations exactly in a single step. Thus, iteration 1 immediately moves x and y to the correct feasible values ($x = 0.5, y = 0.5$).
- Lagrange Multiplier Update:** In the first iteration, λ was updated to -1.0 . In the second iteration, with x and y fixed at the optimal point, the algorithm simply adjusted λ to the correct value of $-e^{0.5} \approx -1.6487$ to satisfy the gradient condition $\nabla L = 0$.

Question 14: Step-Size Strategies in Gradient Descent

- (a) Briefly explain two step-size selection methods (e.g., Backtracking/Armijo, Wolfe conditions, or adaptive methods such as AdaGrad/RMSProp).
- (b) Using the Rosenbrock problem (from Q6) with $x_0 = [-1.2, 1]^\top$, implement GD with the two methods from (a).
- (c) Compare results (e.g., trajectories, function values vs. iterations, convergence behavior).

8.1 Step Size Selection Strategies

- **Armijo Strategy:** This strategy ensures that the function value decreases sufficiently, preventing steps that are too large.

The condition is given by:

$$f(x_k - \alpha \nabla f(x_k)) \leq f(x_k) - c\alpha \|\nabla f(x_k)\|^2$$

Where:

- c : A small constant (usually 10^{-4}).
- α : Learning rate. First, we initialize it as a large number and then decrease it by a constant factor if the condition is not met.

Backtracking Logic: If the condition fails, meaning:

$$f(x_k - \alpha \nabla f(x_k)) > f(x_k) - c\alpha \|\nabla f(x_k)\|^2$$

then the change in the function is too large (step is too big), so we decrease the step size by reducing α by a scaling factor (e.g., $\alpha \leftarrow \rho \alpha$).

- **Wolfe Conditions:** This consists of two rules. The first rule is similar to the Armijo rule (Sufficient Decrease Condition), which is:

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^\top p_k$$

Second Condition: Curvature Condition

This condition prevents taking short steps. The inequality is given by:

$$\nabla f(x_k + \alpha p_k)^\top p_k \geq c_2 \nabla f(x_k)^\top p_k$$

This ensures that if we take a step and the slope along the direction p_k is still very steep (highly negative), we should increase our step size until we reach a point with a smaller slope (closer to zero, indicating we are closer to the minimum).

• AdaGrad

This algorithm is typically used for Stochastic Gradient Descent (SGD). AdaGrad calculates an adaptive step size based on the history of the gradients.

In this algorithm:

- Features that occur frequently (and thus have large accumulated gradients) have their learning rate **reduced** to stabilize convergence.
- Features that are rare (small accumulated gradients) maintain a higher effective learning rate to ensure they are updated sufficiently.

We define the accumulated sum of squared gradients G_t as:

$$G_t = \sum_{i=1}^t g_i^2$$

Where g_i is the gradient at time step i .

Update Rule:

$$x_{t+1} = x_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot g_t$$

8.1 (c) Comparison of Results

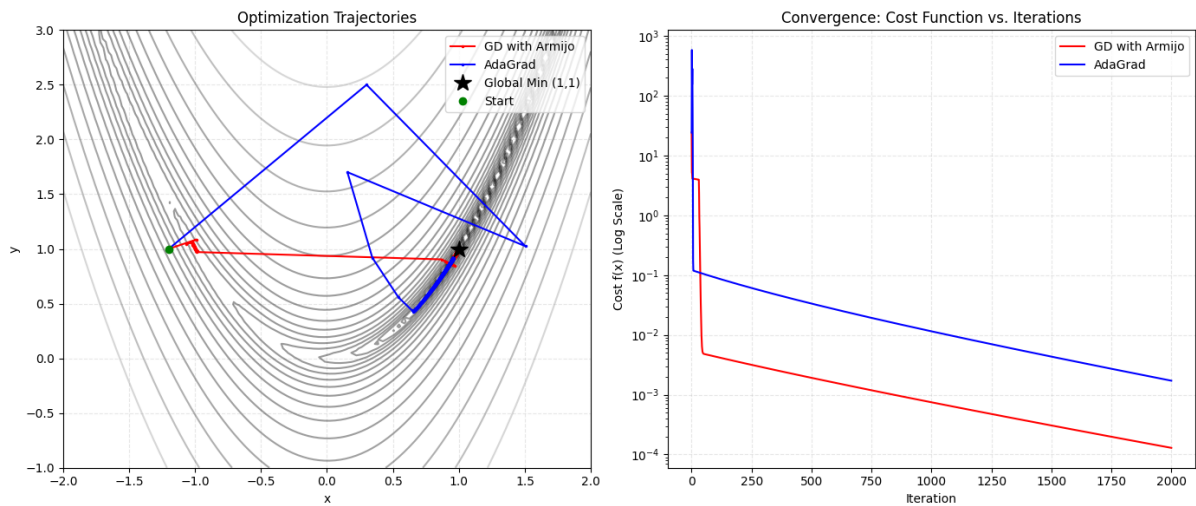


Figure 1: Optimization trajectories (left) and cost convergence (right) for Armijo vs. AdaGrad.

1. Trajectories (Visual Analysis)

- **GD with Armijo (Red Line):** The trajectory is smooth and efficient. It moves immediately towards the valley and follows the floor intimately to the global minimum. The Backtracking strategy prevents overshooting by checking for sufficient decrease at every step.

- **AdaGrad (Blue Line):** The trajectory exhibits significant oscillation and zigzagging. It overshoots the valley floor multiple times. Because AdaGrad adapts based on accumulated gradients without a line search, it takes "blind" steps that result in bouncing off the walls of the Rosenbrock valley.

2. Convergence Behavior

- **Initial Drop:** Armijo shows a massive, vertical drop almost instantly, confirming the line search found an optimal step size immediately. AdaGrad drops quickly but hits a brief plateau.
- **Final Accuracy:** Armijo converged to a cost of $\approx 1.3 \times 10^{-4}$ (very close to 0), while AdaGrad only reached $\approx 1.7 \times 10^{-3}$.

Conclusion: GD with Armijo Backtracking performed better for this problem. AdaGrad's learning rate decayed too aggressively (due to accumulating squared gradients), causing it to "run out of steam" on the flat valley floor before reaching the precise global minimum.